

CSE445 Machine Learning — Final Exam Prep

Dimensionality Reduction (PCA) + Convolutional Neural Networks (CNN)

LECTURE 12: DIMENSIONALITY REDUCTION (PCA)

Part A — Linear Algebra Review (the prerequisites PCA is built on)

1. Linear Transformation

A linear transformation $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a function satisfying:

Additivity: $T(u) + T(v) = T(u + v)$

Homogeneity: $T(cv) = cT(v)$

Every linear transformation can be written as a **matrix multiplication**: $T(v) = Av$, where $A \in \mathbb{R}^{m \times n}$ and $v \in \mathbb{R}^n$.

Example: $A_1 = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $A_2 = \begin{bmatrix} 2 & 1 \\ 1 & -2 \end{bmatrix}$, applied to $v_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $v_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$:

$$A_1 v_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, A_1 v_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$
$$A_2 v_1 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, A_2 v_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

This shows how a matrix "moves" basis vectors — the core visual intuition behind everything that follows.

2. Determinant

The **determinant** of a square matrix is a scalar computed from its elements.

Geometric meaning: it's the **factor by which area (or volume) is scaled** by the transformation.

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

Example: $\det(A_1) = 2 \times 2 - 1 \times 1 = 3$ (area scaled $\times 3$); $\det(A_2) = 2 \times (-2) - 1 \times 1 = -5$ (area scaled $\times 5$, **sign negative $\rightarrow$ orientation flipped**, i.e., v_2 was to the right of v_1 before the transform but ends up to the left afterward).

Special case: if $\det(A) = 0$, the transformation **flattens** all vectors onto a line (or point) — area becomes 0 (e.g., $A_3 = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$, $\det = 0$).

3. Eigenvectors and Eigenvalues

When a matrix A is applied to a vector, most vectors get **"knocked off" their span** (their direction changes).

Eigenvectors are the special vectors that **stay on their own span** — they only get **scaled**, not rotated, by A .

The scaling factor for an eigenvector is its **eigenvalue** λ . The **sign of** λ tells you whether the scaling kept the same direction (+) or flipped it (-).

Defining equation: $Av = \lambda v$

Rearranged: $Av - \lambda Iv = 0 \Rightarrow (A - \lambda I)v = 0$ (where I is the identity matrix)

For a nonzero solution v to exist, we need $\boxed{\det(A - \lambda I) = 0}$ — this equation is solved to find the eigenvalues λ ; each λ is then substituted back to solve for its corresponding eigenvector v .

4. Covariance Matrix (Σ)

Covariance measures how much two random variables vary *together*.

After **centering the data** (subtracting the mean), Σ captures how every feature varies with every other feature:

$$\Sigma = \begin{bmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) \\ \text{Cov}(x_1, x_2) & \text{Var}(x_2) \end{bmatrix} \quad (\text{2D case})$$

Diagonal entries = variance of each individual feature. **Off-diagonal entries** = covariance between feature pairs — larger magnitude means the features change together more.

Sign interpretation:

Positive covariance $\rightarrow$ features are **directly proportional** (one increases, the other tends to increase too)

Negative covariance $\rightarrow$ features are **indirectly (inversely) proportional**

Zero covariance $\rightarrow$ the two features are **independent**

Key property: a covariance matrix is always **symmetric** $\rightarrow$ its **eigenvectors are always perpendicular (orthogonal) to each other**. This is *the* fact that makes PCA work — it

guarantees the principal component axes form a valid, non-overlapping new coordinate system.

Geometric picture: applying Σ (as a linear transformation) to a unit circle stretches it into an **ellipse**. Most points on the circle get knocked off their span, but exactly **two vectors stay on their span — the eigenvectors** v_1, v_2 of Σ . Their eigenvalues λ_1, λ_2 tell you how much each eigenvector direction got stretched — the eigenvector with the **larger eigenvalue points along the direction of greatest variance** in the data.

Part B — Dimensionality Reduction: The Concept

1. Introduction

Datasets with too many features can cause:

Slow computation

Overfitting (poor generalization to new data)

Dimensionality reduction aims to:

Reduce the number of input features

Retain essential information

Convert data into a **lower-dimensional space** while preserving key details

Example: Predicting house prices with bedrooms, square footage, location, etc. Adding too many extra features (room condition, flooring type, ...) makes the dataset large/complex — dimensionality reduction simplifies it without losing much predictive power.

2. Advantages

Faster computation — fewer features speed up training/testing, especially on large datasets

Improved visualization — 2D/3D data is easy to plot, revealing patterns/clusters that are hard to see in high dimensions

Reduced overfitting — fewer features reduce the model's chance of memorizing training data, improving generalization

3. Disadvantages

Potential data loss — important information may be discarded, possibly hurting accuracy

Difficult component selection — choosing how many dimensions to keep is tricky: too few → lose key data; too many → keep noise and risk overfitting

Part C — Principal Component Analysis (PCA)

1. The Core Idea

Analogy: Imagine photographing a group of people from different camera angles.

If people, when projected onto the camera's line of sight, appear **close together**, the photo captures **less detail** (low variance).

If they appear **spread out**, the photo captures **more detail** (high variance).

PCA = taking a "picture" of your data from the optimal angle, i.e., finding the projection line where the data — once projected onto it — has **maximum variance (most spread out)**. **PCA finds the line that maximizes variance.**

2. Why Eigenvectors of the Covariance Matrix?

Applying the covariance matrix Σ (as a linear transformation) to a unit circle produces an ellipse. The **eigenvectors of Σ** are the two directions that don't get knocked off their span — and they turn out to be exactly the directions of **maximum and minimum variance** in the data. Since Σ is symmetric, these eigenvectors are **guaranteed to be perpendicular**.

The eigenvector with the **larger eigenvalue** = direction of **most variance** → this becomes **Principal Component 1 (PC1)**.

The eigenvector with the **smaller eigenvalue** = direction of **less variance** → **PC2** (and so on for higher dimensions).

3. Step-by-Step PCA Procedure (conceptual)

Mean-shift (center) the data: $x_i^{centered} = x_i - \mu_i$ for every feature, so the data is centered at the origin.

(Optional) Scale the data so feature magnitudes lie roughly between -1 and +1 — optional, depends on the original scale of your features.

Compute the covariance matrix Σ of the centered data.

Eigen-decompose Σ : solve $\det(\Sigma - \lambda I) = 0$ for eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$, then solve $(\Sigma - \lambda_i I)v_i = 0$ for each corresponding eigenvector v_i (normalize each v_i to unit length).

Rotate the data onto the new eigenvector axes: $X_{PCA} = X_{centered} V$, where $V = [v_1, v_2, \dots]$ (eigenvectors as columns, ordered by descending eigenvalue). This "rotates" the data so its axes now align with the directions of maximum variance.

Select components: the rotated axes are the **Principal Components (PCs)**, arranged in **descending order of eigenvalue**. Keep the top PCs that capture "enough" variance and discard the rest — **this is the actual dimensionality reduction step.**

4. Weight of Each Principal Component (Explained Variance Ratio)

$$w_{\lambda_i} = \frac{\lambda_i}{\lambda_1 + \lambda_2 + \dots + \lambda_m}$$

This tells you **what fraction of the total variance** each PC explains. Example: if $\lambda_1 = 5.7$, $\lambda_2 = 0.3$:

$$w_{\lambda_1} = \frac{5.7}{6.0} = 0.95, \quad w_{\lambda_2} = \frac{0.3}{6.0} = 0.05$$

PC1 explains 95% of the variance, PC2 only 5% → we can keep just PC1 for model training and discard PC2, reducing the dimensionality from 2 → 1 while losing very little information.

Part D — Full Worked Numerical Example (PCA by hand)

Given data X (4 samples, 2 features):

$$X = \begin{bmatrix} 2.5 & 2.4 \\ 0.5 & 0.7 \\ 2.2 & 2.9 \\ 1.9 & 2.2 \end{bmatrix}$$

Step 1 — Mean Centering

$$\mu = \left[\frac{2.5 + 0.5 + 2.2 + 1.9}{4}, \frac{2.4 + 0.7 + 2.9 + 2.2}{4} \right] = [1.775, 2.05]$$

$$X_{centered} = X - \mu = \begin{bmatrix} 0.725 & 0.35 \\ -1.275 & -1.35 \\ 0.425 & 0.85 \\ 0.125 & 0.15 \end{bmatrix}$$

Step 2 — Covariance Matrix

$$\Sigma = \frac{1}{n-1} X_{centered}^T X_{centered} = \frac{1}{3} \begin{bmatrix} 1.0675 & 1.1225 \\ 1.1225 & 1.2975 \end{bmatrix} = \begin{bmatrix} 0.3558 & 0.3742 \\ 0.3742 & 0.4325 \end{bmatrix}$$

Step 3 — Eigen-decomposition

Set $\det(\Sigma - \lambda I) = 0$:

$$(0.3558 - \lambda)(0.4325 - \lambda) - (0.3742)^2 = 0 \implies \lambda^2 - 0.7883\lambda + 0.0138 = 0$$

Quadratic formula: discriminant $= 0.7883^2 - 4(0.0138) = 0.5662$, $\sqrt{0.5662} \approx 0.7525$

$$\lambda_{1,2} = \frac{0.7883 \pm 0.7525}{2} \implies \lambda_1 \approx 0.7704, \lambda_2 \approx 0.0179$$

Eigenvector for $\lambda_1 \approx 0.7704$:

$$\Sigma - \lambda_1 I \approx \begin{bmatrix} -0.4146 & 0.3742 \\ 0.3742 & -0.3379 \end{bmatrix}$$

Solve $-0.4146v_x + 0.3742v_y = 0 \implies v_x/v_y = 0.903$. Normalizing: $v_1 = \begin{bmatrix} 0.672 \\ 0.744 \end{bmatrix}$

Eigenvector for $\lambda_2 \approx 0.0179$:

$$\Sigma - \lambda_2 I \approx \begin{bmatrix} 0.3379 & 0.3742 \\ 0.3742 & 0.4146 \end{bmatrix}$$

Solve $\implies v_x/v_y = -1.107$. Normalizing: $v_2 = \begin{bmatrix} -0.742 \\ 0.671 \end{bmatrix}$

(Sanity check: $v_1 \cdot v_2 \approx 0 \implies$ they are perpendicular $\checkmark$, as guaranteed by Σ 's symmetry.)

Step 4 — Project (rotate) the data onto the PCs

$$V = [v_1, v_2], \quad X_{PCA} = X_{centered} V = \begin{bmatrix} -1.275 & -1.35 \\ 0.425 & 0.85 \\ 0.125 & 0.15 \\ 0.725 & 0.35 \end{bmatrix} \begin{bmatrix} 0.672 & -0.742 \\ 0.744 & 0.671 \end{bmatrix}$$

$$X_{PCA} \approx \begin{bmatrix} -1.85 & 0.01 \\ 0.91 & 0.27 \\ 0.20 & 0.01 \\ 0.01 & \dots \end{bmatrix} \quad (\text{columns} = PC_1, PC_2)$$

Interpretation: the data is now expressed in a new basis where the first column (**PC1**) carries almost all the spread, and the second column (**PC2**) is nearly flat (very little variance) — visually the ellipse is now aligned exactly with the horizontal (PC1) and vertical (PC2) axes.

Explained variance: $w_{\lambda_1} = 0.7704 / (0.7704 + 0.0179) \approx 0.977$, $w_{\lambda_2} \approx 0.023 \rightarrow$ **PC1 alone captures ~97.7% of the variance**, so for dimensionality reduction we'd keep only PC1 and drop PC2, reducing 2 features down to 1 with minimal information loss.

Part E — Limitations of PCA (Summary)

PCA is a **linear** dimensionality reduction technique — it finds directions (principal components) that **maximize linear variance**.

PCA transforms features into PCs that are **not guaranteed to align with the most predictive features** — **PCA maximizes variance, not predictive power**. It may keep high-variance components that are actually less useful for prediction, and could discard a low-variance feature that was actually important for the label.

If the data lies on a **nonlinear manifold** (spiral, circle, curved surface), PCA will **fail to capture the true structure** and likely **discard important nonlinear information** (this is why nonlinear techniques like t-SNE, UMAP, or Kernel PCA exist).

LECTURE 13: CONVOLUTIONAL NEURAL NETWORKS (CNN)

1. Introduction

A **CNN (Convolutional Neural Network)** is a neural network **specialized for grid-like data**, especially images (pixels on a 2D grid).

Key properties:

Automatic feature extraction — learns edges, textures, shapes, objects directly from raw pixels (no manual feature engineering)

Local connectivity — each neuron "looks" at only a small region of the image (its **receptive field**) instead of the whole image at once

Weight sharing — the *same* filter (set of weights) is slid across the entire image $\rightarrow$ far **fewer parameters**, more efficient than a fully-connected network on images

Hierarchical features — early layers learn simple patterns (edges); deeper layers learn increasingly complex/abstract concepts (parts → objects)

Typical applications: image classification (cat vs. dog), object detection & segmentation (self-driving cars, medical imaging), face recognition, OCR, and other vision tasks.

2. Image Types

Type	Channels	Pixel representation	Typical shape
RGB image	3 (Red, Green, Blue)	Triplet (R, G, B)	$H \times W \times 3$
B&W / Grayscale	1 (intensity only)	Single value	$H \times W$ (or $H \times W \times 1$)

Exam tip / watch out: one slide in the deck lists " $d = 1$ if RGB, $d = 3$ if grayscale" — this is backwards from the correct convention (confirmed by the "Shapes and Notations" slide and the RGB-convolution diagram elsewhere in the same lecture): **grayscale → 1 channel, RGB → 3 channels**. Know the correct convention for the exam.

3. The Idea of a CNN — Convolution

Convolution = "slide" the **filter/kernel** over the image spatially, computing a **dot product** at each position.

If the filter is size $F \times F \times d$ and the image is $N \times N \times d$ (same depth d as the filter), the output ("activation map") has size $M \times M$, where:

$$M = \frac{N - F}{S} + 1$$

(S = **stride**, the number of pixels the filter moves each step.)

Worked Example: 7×7 image, 3×3 filter

Stride = 1: $M = (7 - 3)/1 + 1 = 5 \rightarrow$ a 5×5 output. (Verified by the slide's 25 slide-positions of the filter.)

Stride = 2: $M = (7 - 3)/2 + 1 = 3 \rightarrow$ a 3×3 output.

Stride = 3: $M = (7 - 3)/3 + 1 = 2.33 \rightarrow$ **not an integer!** The filter **does not fit** cleanly — you cannot apply a 3×3 filter to a 7×7 input with stride 3 without padding.

Zero Padding

Common solution when the filter doesn't fit cleanly (or when you want to preserve spatial size): **pad the border of the image with zeros**.

E.g., padding the 7×7 image with a 1-pixel border of zeros makes it effectively 9×9 , letting a stride-3 filter fit, or letting a stride-1 filter produce an output the *same size* as the original input ("same" padding).

The general output-size formula including padding P : $M = \lfloor \frac{N-F+2P}{S} \rfloor + 1$ (the deck writes the no-padding version explicitly: $M = \lfloor (N - F)/S \rfloor + 1$).

4. Activation Map (Feature Map)

Each filter is like a "**pattern detector**" (e.g., detects a vertical edge, a corner, a texture).

The **activation map** shows *where* in the image that pattern appears, and *how strongly*:

Bright/large values $\rightarrow$ "I see my pattern here strongly"

Dark/small values $\rightarrow$ "My pattern is weak or absent here"

Typically, the raw dot-product output is passed through an **activation function** (e.g., **ReLU**) to produce the final activation map.

Worked Example

A 5×5 image with a 3×3 filter, stride 1, produces a $(\frac{5-3}{1} + 1) = 3 \times 3$ activation map.

Each cell of that map = one dot product between the filter and the corresponding 3×3 patch of the image, then passed through ReLU.

5. Convolution on RGB Images

For a color (RGB) image, the input has depth $d = 3$. The filter must **also have depth 3** (i.e., filter shape $F \times F \times 3$) so it can slide across all three channels **simultaneously** — at each spatial position, the filter's dot product sums contributions from all 3 channels, collapsing them into a **single** output value per position (so one filter over an RGB image still gives a 2D $M \times M$ activation map, not $M \times M \times 3$).

If there are **multiple filters** (K of them), each produces its own $M \times M$ activation map, and they're stacked together: the overall output is $A \in \mathbb{R}^{M \times M \times K}$.

6. Mathematical Formulation of CNN (Formal Notation)

Input image/feature map: $X \in \mathbb{R}^{N \times N \times d}$, where $N \times N$ is spatial size and d is depth/channels ($d = 1$ grayscale, $d = 3$ RGB).

Filter/Kernel: $W \in \mathbb{R}^{F \times F \times d}$ — same depth d as the input.

Stride S = pixels moved per step.

Activation map (for ONE filter, no padding): $A \in \mathbb{R}^{M \times M}$, with $M = \lfloor (N - F)/S \rfloor + 1$.

For each patch, flatten the input patch $X_{patch} \in \mathbb{R}^{F \times F \times d}$ into $X_{flatten} \in \mathbb{R}^{F^2 d}$, and flatten the filter similarly into $W_{flatten} \in \mathbb{R}^{F^2 d}$.

Pre-activation value at location (i, j) : $z_{ij} = W_{flatten}^T X_{flatten} \in \mathbb{R}$ (a plain dot product).

After activation function σ : $A_{ij} = \sigma(z_{ij})$.

With K filters: the full output is $A \in \mathbb{R}^{M \times M \times K}$ (one activation map per filter, stacked along a new depth dimension).

7. Pooling Layers (Downsampling)

Pooling **downsamples** feature maps — reduces their spatial size — using a small sliding window (e.g., $P \times P$), analogous in mechanism to a convolution filter sliding over the map, but without learnable weights.

Types of Pooling

Type	Rule	Notes
Max Pooling	output = max of all values in the $P \times P$ window	Keeps the strongest activation (strongest evidence of a feature). More commonly used in modern CNNs
Average Pooling	output = average of all values in the window	Smooths values; less used in modern CNNs

Why Pooling?

Reduces computation & parameters in later layers (smaller feature maps to process)
Acts as a form of **regularization**, helping **prevent overfitting**

8. Fully-Connected (FC) Layers

Acts like a **standard MLP (feedforward neural network)** layer, typically placed at the **end** of the CNN.

Division of labor in a CNN:

Convolution + pooling layers → learn **spatial features** (edges, textures, shapes, objects)

Fully-connected layers → take those learned features and use them to make the **global decision** (e.g., "Is this a cat, dog, or car?")

9. Example CNN Architecture (classic digit-classification pipeline)

A typical small CNN for classifying 28×28 grayscale digit images (e.g., MNIST):



INPUT (28×28×1)

→ Conv_1: 5×5 kernel, valid padding → n1 channels (24×24×n1)
 → Max-Pooling (2×2) → (12×12×n1)
 → Conv_2: 5×5 kernel, valid padding → n2 channels (8×8×n2)
 → Max-Pooling (2×2) → (4×4×n2)
 → Flatten
 → fc_3: Fully-Connected + ReLU activation → n3 units (with dropout)
 → fc_4: Fully-Connected → 10 outputs (digits 0-9)

"Valid padding" means no zero-padding (output shrinks per the $M = (N - F)/S + 1$ formula) — e.g., $28 \rightarrow 24$ with a 5×5 filter, stride 1: $(28 - 5)/1 + 1 = 24$. ✓

Max-pooling 2×2 halves the spatial size each time: $24 \rightarrow 12, 8 \rightarrow 4$.

Flatten reshapes the final $4 \times 4 \times n_2$ tensor into a 1D vector to feed into the fully-connected layers.

Dropout is used in the FC layer as regularization to reduce overfitting.

Final layer has 10 output neurons (one per digit class 0-9).

10. Example of Inference (Visualizing what layers "see")

Feeding a photo of a puppy through a deep CNN (17 convolution layers, in the example) and visualizing the resulting activation maps at different depths shows:

Early layers are usually **more interpretable** and **preserve most of the low-level information** in the input image (edges, simple shapes still visually resemble the dog).

Deeper layers: features become **less visually intuitive and more abstract** — the network **focuses on class-specific patterns while discarding more general image details** (the visualizations no longer "look like" a dog to a human, but encode information the network uses to decide the class).

11. CNNs — Backbone of Modern Vision

CNNs learn **hierarchical visual features** (edges → textures → parts → objects) directly from pixels, making them the standard backbone for most computer vision systems:

Task	Example Models	What they do
Image classification	ResNet, VGG	"Is this a cat, dog, or car?" (single label for whole image)
Object detection	Faster R-CNN, YOLO, SSD	Draw bounding boxes + labels around objects in an image
Semantic & instance segmentation	U-Net, Mask R-CNN	Classify every pixel (e.g., road vs. car vs. pedestrian)

PROBABLE EXAM QUESTIONS & ANSWERS

A. Conceptual / Definition-style

Q1. What is a linear transformation, and how can it always be represented? A: A function $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ satisfying additivity ($T(u) + T(v) = T(u + v)$) and homogeneity ($T(cv) = cT(v)$). Every linear transformation can be represented as a matrix multiplication $T(v) = Av$.

Q2. What does the determinant of a matrix represent geometrically? A: The factor by which area (2D) or volume (higher-D) is scaled by the transformation. A negative determinant means the transformation also flips orientation. A determinant of 0 means the transformation flattens all vectors onto a line (or a point) — it destroys a dimension.

Q3. Define eigenvector and eigenvalue. A: An eigenvector of a matrix A is a vector that does not change direction (isn't "knocked off its span") when A is applied to it — it's only scaled. The scale factor is its eigenvalue λ , satisfying $Av = \lambda v$. Eigenvalues are found by solving $\det(A - \lambda I) = 0$.

Q4. Why are the eigenvectors of a covariance matrix always perpendicular to each other? A: Because a covariance matrix is always **symmetric**, and the eigenvectors of any symmetric matrix are guaranteed to be orthogonal (perpendicular).

Q5. What does a positive, negative, and zero covariance mean? A: Positive $\rightarrow$ the two features are directly proportional (increase together). Negative $\rightarrow$ indirectly/inversely proportional (one increases as the other decreases). Zero $\rightarrow$ the two features are independent of each other.

Q6. What problem does dimensionality reduction solve? A: Too many features can cause slow computation and overfitting. Dimensionality reduction reduces the number of input features while retaining essential information, converting data to a lower-dimensional space.

Q7. List the advantages and disadvantages of dimensionality reduction. A: Advantages: faster computation, improved visualization (2D/3D plots reveal patterns), reduced overfitting. Disadvantages: potential loss of important information, and difficulty choosing the right number of dimensions to keep (too few loses key data, too many keeps noise).

Q8. Explain the core idea behind PCA using the "photograph from an angle" analogy. A: Just as a photo captures more detail when the subjects are spread out (not squished together) as seen from the camera's line of sight, PCA looks for the projection line ("angle") onto which the data, once projected, is most spread out (has maximum variance). This line is the direction that preserves the most information about the original data.

Q9. How does PCA connect eigenvectors/eigenvalues of the covariance matrix to the actual principal components? A: The eigenvectors of the data's covariance matrix Σ point in the directions of maximum and minimum variance of the data. The eigenvector with the largest eigenvalue is the direction of greatest spread — this becomes PC1. The next-largest eigenvalue's eigenvector (orthogonal to PC1, since Σ is symmetric) becomes PC2, and so on.

Q10. Outline the full step-by-step PCA algorithm. A: (1) Mean-center the data (subtract each feature's mean); (2) optionally scale features; (3) compute the covariance matrix Σ ; (4) find Σ 's eigenvalues (solve $\det(\Sigma - \lambda I) = 0$) and corresponding eigenvectors; (5) project/rotate the centered data onto the eigenvectors ($X_{\text{PCA}} = X_{\text{centered}} \times V$, V = eigenvectors sorted by descending eigenvalue); (6) keep the top-k principal components (largest eigenvalues) and discard the rest.

Q11. How do you decide how much variance each principal component explains? A: Compute the weight $w_{\lambda_i} = \lambda_i / \sum_j \lambda_j$ for each eigenvalue — this gives the fraction of total variance explained by that component. Components are always ordered by descending eigenvalue (PC1 explains the most variance).

Q12. What are the limitations of PCA? A: It's a linear technique — it only captures linear variance, so it fails on data lying on a nonlinear manifold (spiral, circle, curved surface), likely discarding important nonlinear structure. Also, PCA maximizes variance, not predictive power — the components it selects (high variance) aren't guaranteed to be the most useful for a prediction task; it might discard a low-variance but highly predictive feature.

Q13. What makes a CNN different from a standard fully-connected neural network for image data? A: CNNs exploit local connectivity (each neuron looks at only a small receptive field, not the whole image), weight sharing (the same filter slides across the whole image, drastically reducing parameters), and hierarchical feature learning (early layers learn simple patterns like edges, deeper layers learn complex/abstract concepts).

Q14. What is the difference between an RGB image and a grayscale image, in terms of channels and shape? A: RGB has 3 channels (Red, Green, Blue), pixel = (R,G,B) triplet, shape $H \times W \times 3$. Grayscale has 1 channel (intensity only), single value per pixel, shape $H \times W$ (or $H \times W \times 1$).

Q15. What is a filter/kernel in a CNN, and what does "sliding" mean? A: A filter is a small weight matrix (e.g., 3×3) that acts as a pattern detector. "Sliding" means moving this filter across the image spatially, computing a dot product between the filter and the underlying image patch at each position, to produce the activation map.

Q16. Give the formula for output (activation map) size, and define each symbol. A: $M = \frac{N-F}{S} + 1$, where N = input spatial size, F = filter size, S = stride (number of pixels moved per step), M = output spatial size.

Q17. Why is zero-padding used? A: To let the filter "fit" cleanly when the raw formula would give a non-integer output size (e.g., 7×7 input, 3×3 filter, stride 3 $\rightarrow$ 2.33, invalid), and/or to control/preserve the spatial size of the output (e.g., "same" padding keeps output size equal to input size).

Q18. What does an activation map represent, and what does a bright vs. dark value mean? A: It's the output of a convolution + activation function for one filter — it shows where in the image the filter's pattern appears and how strongly. Bright/large values mean the pattern is strongly present at that location; dark/small values mean it's weak or absent.

Q19. How does convolution work on an RGB (3-channel) image? A: The filter must have the same depth as the input (3, for RGB), i.e., shape $F \times F \times 3$. At each spatial position, the filter's dot product sums over all 3 channels simultaneously, producing a single output value per position — so one filter still yields a 2D $M \times M$ activation map (not $M \times M \times 3$). With K filters, the outputs stack into an $M \times M \times K$ volume.

Q20. Describe the two types of pooling, and state which is more commonly used. A: Max pooling takes the maximum value in each $P \times P$ window (keeps the strongest activation) — more commonly used in modern CNNs. Average pooling takes the average value in each window (smooths values) — less used in modern CNNs.

Q21. Why use pooling layers? A: To reduce computation and the number of parameters in later layers (by shrinking feature maps), and to act as a form of regularization that helps prevent overfitting.

Q22. What role do fully-connected (FC) layers play at the end of a CNN? A: They act like a standard MLP layer. While the convolution+pooling layers learn spatial features (edges, textures, shapes, objects), the FC layers use those extracted features to make the final global decision/classification (e.g., which class the image belongs to).

Q23. In the example CNN architecture, what happens to spatial size as data passes through the network, and why? A: It shrinks through the convolution layers (due to valid/no padding — e.g., $28 \rightarrow 24$ with a 5×5 filter) and shrinks further through max-pooling layers (e.g., $24 \rightarrow 12$ with 2×2 pooling), because pooling and (unpadded) convolution both reduce spatial dimensions, while the number of channels (feature maps) typically increases to capture more complex/varied features.

Q24. What is the general trend observed when visualizing activations from early vs. deep CNN layers? A: Early layers are more interpretable and preserve most low-level information (edges, shapes) that still visually resembles the input. Deeper layers produce increasingly abstract, less visually-intuitive features, focusing on class-specific patterns and discarding general image details.

Q25. Name key CNN-based architectures for classification, detection, and segmentation. A: Classification: ResNet, VGG. Object detection: Faster R-CNN, YOLO, SSD (draw bounding boxes + labels). Segmentation: U-Net, Mask R-CNN (classify every pixel).

B. Broad / "Explain in detail" style

Q26. Explain, step by step with the given numerical example, how PCA reduces a 2-feature dataset to 1 feature. A: (Full worked example from Part D.) Starting from 4 samples with 2 features, first center the data by subtracting each feature's mean. Compute the covariance matrix from the centered data. Solve $\det(\Sigma - \lambda I) = 0$ to get eigenvalues (here $\lambda_1 \approx 0.7704$, $\lambda_2 \approx 0.0179$) and their eigenvectors ($v_1 \approx [0.672, 0.744]$, $v_2 \approx [-0.742, 0.671]$, which are perpendicular). Project the centered data onto these eigenvectors ($X_{\text{PCA}} = X_{\text{centered}} \cdot V$) to get the data expressed in the new PC1/PC2 basis. Since λ_1 is much larger than λ_2 (explaining $\sim 97.7\%$ vs $\sim 2.3\%$ of variance), nearly all the information is captured along PC1 alone — so we can drop the PC2 column entirely, reducing the dataset from 2 dimensions to 1 with minimal information loss.

Q27. Walk through how convolution works, including how output size and padding are determined, using the 7×7 image / 3×3 filter example. A: The 3×3 filter slides over the 7×7 image, and at every valid position we compute a dot product between the filter weights and the underlying image patch. With stride 1, the filter can occupy 5 positions per row/column (since it needs a 3-wide window inside a 7-wide row), producing a 5×5 activation map, matching $M = (7 - 3) / 1 + 1 = 5$. With stride 2, it produces a 3×3 map ($M = 3$). With stride 3, the formula gives $M = 2.33$ — not a whole number, meaning the filter doesn't

tile the image evenly and simply won't fit past a certain point. The standard fix is zero-padding the image's border (e.g., padding 7×7 up to 9×9) so that the sliding filter fits evenly for the desired stride, or so that the output size can be controlled independently (e.g., kept equal to the input size).

Q28. Describe the complete pipeline of a CNN using the example digit-classification architecture, and explain each stage's purpose.

A: An input grayscale digit image ($28 \times 28 \times 1$) first passes through a convolutional layer (5×5 filters, valid/no padding) producing n_1 feature maps of size 24×24 (using $M = (28 - 5) / 1 + 1 = 24$) — this stage extracts low-level spatial features like edges. A 2×2 max-pooling layer then halves the spatial size to 12×12 , reducing computation while keeping the strongest activations. A second convolution (5×5 , valid padding) extracts higher-level features into n_2 channels at 8×8 , followed by another 2×2 max-pool down to 4×4 — deeper layers combine simple features into more complex patterns. The resulting $4 \times 4 \times n_2$ volume is flattened into a 1D vector and fed into fully-connected layers (with ReLU activation and dropout for regularization), which combine all the extracted spatial features to make the final global classification decision — outputting 10 values (one per digit class 0–9).

Q29. Explain how a single convolutional filter processes a multi-channel (RGB) input, and how this changes when there are multiple filters. A: A convolution filter applied to an RGB image must span all input channels — i.e. the filter has shape $F \times F \times 3$, matching the image's depth of 3. As the filter slides spatially across the image, at each position it computes one dot product that sums contributions from all three channels together, collapsing them into a single scalar value per spatial location. Consequently, a single filter over an RGB image produces one $2D$ $M \times M$ activation map — not three separate ones. If K different filters are used (each independently detecting a different pattern, e.g., one for vertical edges, one for a certain texture), each produces its own $M \times M$ activation map; stacking all K of these along a new depth dimension gives the layer's full output volume $A \in \mathbb{R}^{(M \times M \times K)}$.

C. Short / "brief" style (rapid-fire)

Q: Formula for output size after convolution (no padding)? A: $M = (N - F) / S + 1$

Q: What does $\det(A)=0$ mean geometrically? A: The transformation flattens all vectors onto a line/point (destroys a dimension); the scaled area becomes zero.

Q: Defining equation for eigenvectors/eigenvalues? A: $Av = \lambda v$, found via $\det(A - \lambda I) = 0$.

Q: Are eigenvectors of a covariance matrix always orthogonal? A: Yes — because covariance matrices are always symmetric.

Q: What does PCA maximize? A: Variance of the data when projected onto the chosen axis/axes (not predictive power).

Q: Which eigenvector becomes PC1? A: The one with the largest eigenvalue.

Q: Formula for the fraction of variance explained by PC i ? A: $w_{\lambda_i} = \lambda_i / \sum_j \lambda_j$

Q: Main limitation of PCA on nonlinear data? A: It's a linear method, so it fails to capture nonlinear manifold structure (spirals, circles, curved surfaces).

Q: Channels for grayscale vs RGB? A: Grayscale = 1 channel; RGB = 3 channels.

Q: What is weight sharing in a CNN? A: The same filter (weights) is used across the entire image, drastically reducing the number of parameters.

Q: What does a filter detect? A: A specific pattern (e.g., an edge, corner, or texture) — it's a "pattern detector."

Q: Formula relating filter depth to input depth? A: They must match — filter shape is $F \times F \times d$, same d as the input.

Q: Which pooling type is more common today? A: Max pooling.

Q: Two benefits of pooling? A: Reduces computation/parameters; acts as regularization against overfitting.

Q: What do FC layers do at the end of a CNN? A: Use the extracted spatial features to make the final classification decision (like a standard MLP).

Q: Do CNN features get more or less interpretable in deeper layers? A: Less interpretable / more abstract, and more class-specific.

QUICK-REVIEW CHEAT SHEET

Concept	Formula / Key Fact
Linear transformation	$T(v) = Av$; additive & homogeneous
Determinant (2x2)	$\det = ad - bc$; = area/volume scale factor; sign flip = orientation flip; 0 = flattens to a line
Eigen-equation	$Av = \lambda v \Rightarrow \det(A - \lambda I) = 0$
Covariance matrix (2D)	$\Sigma = \begin{bmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) \\ \text{Cov}(x_1, x_2) & \text{Var}(x_2) \end{bmatrix}$; always symmetric $\rightarrow$ eigenvectors $\perp$
PCA goal	Find directions (eigenvectors of Σ) that maximize projected variance
Variance explained by PC _{i}	$w_{\lambda_i} = \lambda_i / \sum \lambda_j$
PCA limitation	Linear only; maximizes variance $\neq$ predictive power

Concept	Formula / Key Fact
CNN output size	$M = (N - F)/S + 1$ (add padding term $2P$ in numerator if padded)
CNN filter depth rule	filter depth = input depth (d)
Channels	Grayscale $d = 1$; RGB $d = 3$
Pooling	Max (common) or Average (less common); reduces size + regularizes
CNN layer order	Conv (+ReLU) $\rightarrow$ Pool $\rightarrow$... $\rightarrow$ Flatten $\rightarrow$ Fully-Connected $\rightarrow$ Output

PCA in one line: Center the data, compute its covariance matrix, find that matrix's eigenvectors/eigenvalues, and project the data onto the top eigenvectors (by eigenvalue) — these are the directions of maximal variance, and keeping only the top few gives you a lower-dimensional representation that preserves most of the original information.

CNN in one line: Learnable filters slide over an image computing dot products (convolution) to build activation maps that detect patterns; pooling shrinks these maps and adds robustness; stacked conv+pool layers build increasingly abstract hierarchical features; and final fully-connected layers turn those features into a classification decision.